

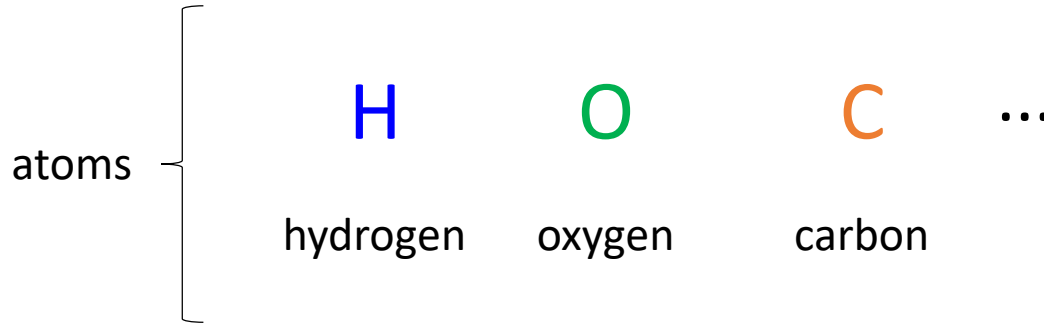
CS360 Introduction to Database

Design Theory for Relational Databases

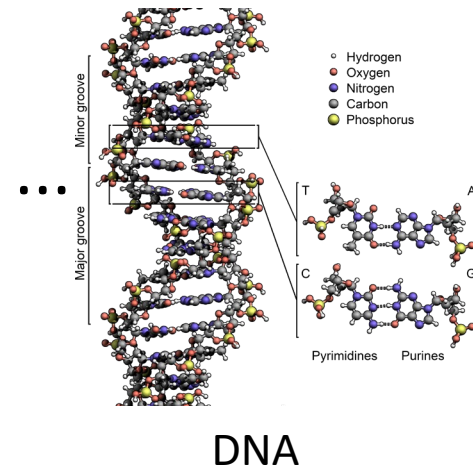
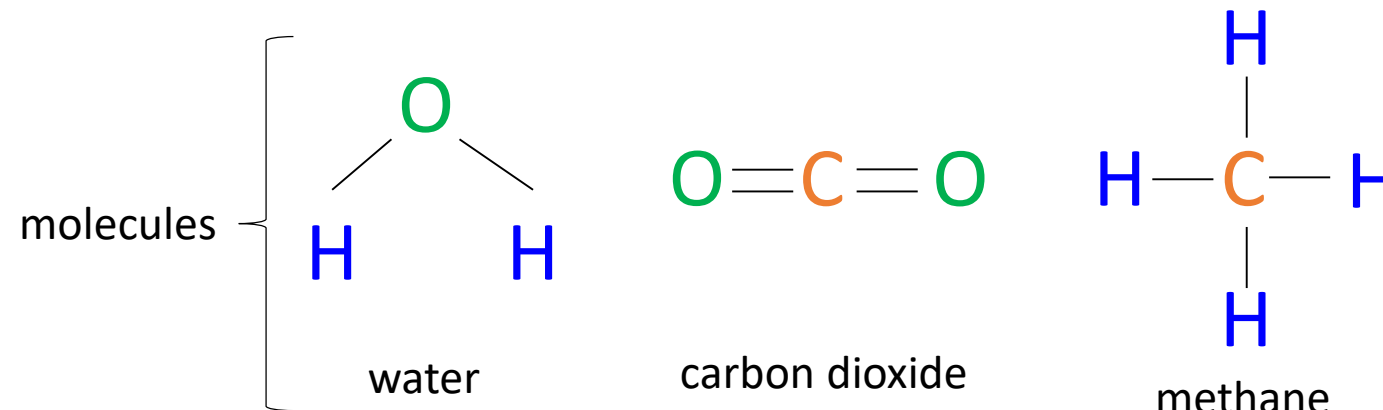
Min-Soo Kim

School of Computing, KAIST

If there is a God who designed all elements ...

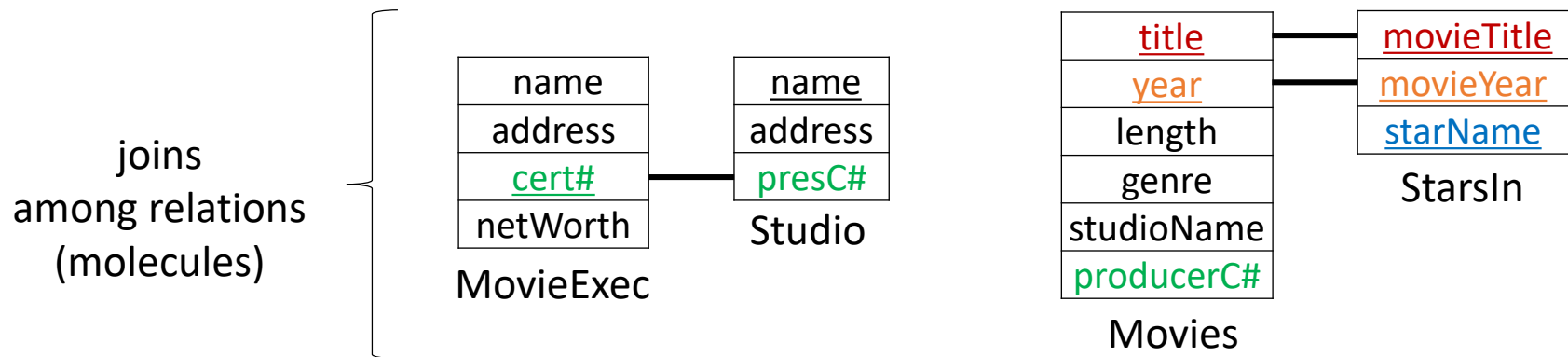
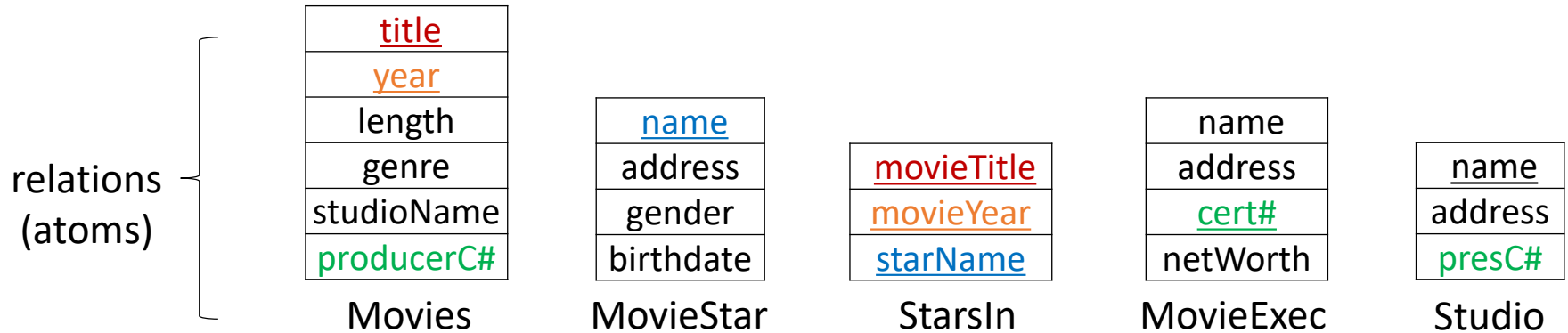


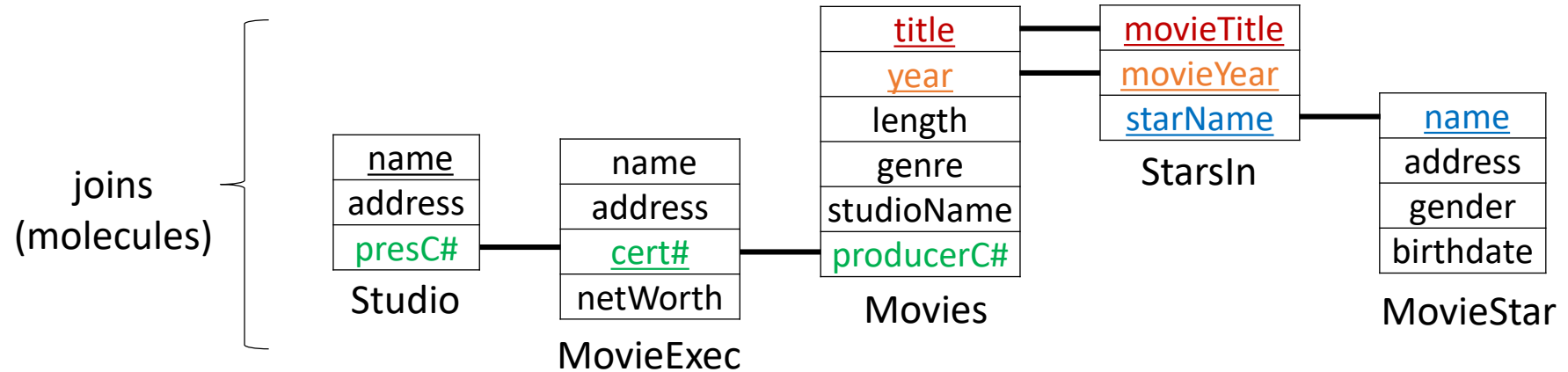
Periodic Table of the Elements



$$|atoms| \ll |molecules|$$

You are a god in the universe of a database





$|relations| \ll |joins|$

Design of database schema (defining the elements) significantly affects how relations are joined (and the performance of all applications)

- a good design allows relations to be joined **properly** and **efficiently**
- a bad design makes relations joined **causing many problems**

Introduction

- Many ways about designing a relational database schema
- It is important to have room to improve an initial schema
 - especially by eliminating redundancy
 - the problems often involve trying to combine too much into one relation
- **Dependencies**: well-developed theory for relational DBs
 - what makes a good relational database schema
 - what we can do about a schema if it has flaws
- **Anomalies**: the problems caused in relation schemas by the presence of certain dependencies

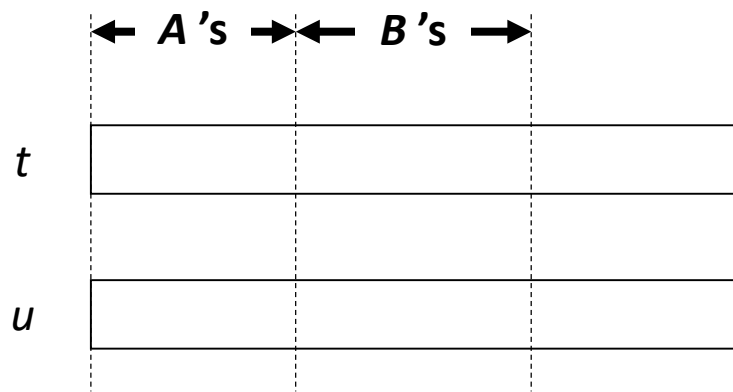
Functional dependencies

Functional dependencies (FD)

- Generalization of the idea of a key for a relation
 - we use the notation of FDs to define normal forms (NF) for relation schemas
- Normalization
 - we decompose a relation into two or more relations in order to remove anomalies
 - tool for improving our designs of relation schemas

Definition of functional dependency (FD)

- FD: $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$
 - if two tuples of R agree on all of the attributes $A_1, A_2, \dots, A_n$
 - then they must also agree on all of another set of attributes $B_1, B_2, \dots, B_m$
 - we say that $A_1, A_2, \dots, A_n$ functionally determine $B_1, B_2, \dots, B_m$
 - note: it is **not necessary** for the A's and B's to appear consecutively or for the A's to precede the B's



relation R

If t and u agree here
Then they must agree here

- Relation R satisfies the FD
 - if every instance of R will be one in which a given FD is true
 - note: constraint on R, not saying something about one particular instance of R, saying something about all possible instances of R
- It is common for right side of an FD to be a single attribute
 - $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ is equivalent to the set of FD's:

$$A_1, A_2, \dots, A_n \rightarrow B_1$$

$$A_1, A_2, \dots, A_n \rightarrow B_2$$

...

$$A_1, A_2, \dots, A_n \rightarrow B_m$$

Example

- **Movies1**(title, year, length, genre, studioName, **starName**)
 - this relation tries to “do too much” (not a good design)
 - original database schema: Movies, Studio, StarsIn, ...
 - following FD **holds**: *title year → length genre studioName*
 - but, this FD does **not hold**: *title year → starName*

title	year	length	genre	studioName	starName
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With the Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

Keys of relations

- One or more attributes $\{A_1, A_2, \dots, A_n\}$ is a key for R if
 - those attributes functionally determine all other attributes
 - i.e., impossible for two distinct tuples of R to agree on all of $A_1, A_2, \dots, A_n$
 - no proper subset of $\{A_1, A_2, \dots, A_n\}$ functionally determine all other attributes
 - i.e., a key must be minimal
- e.g., $\{\text{title}, \text{year}, \text{starName}\}$ form a key for Movies1
 - they functionally determine all the other attributes
 - no proper subset of $\{\text{title}, \text{year}, \text{starName}\}$ functionally determines all other attributes
 - $\{\text{title}, \text{year}\}$ is not a key
 - $\{\text{year}, \text{starName}\}$ is also not a key
 - $\{\text{title}, \text{starName}\}$ is also not a key

- Relation may have more than one key
 - we call all the keys as **candidate keys**
- Common to designate one of the keys as the **primary key**
 - choice of primary key can influence some implementation issues such as how the relation is stored on disk
 - note: theory of FD's gives no special role to primary keys
- **Superkeys**
 - a set of attributes that contains a key
 - thus, every key is a superkey
 - however, some superkeys are not (minimal) keys
 - satisfy the first condition of a key
 - not satisfy the second condition of a key (minimality)
 - e.g., {title, year, starName, length, studioName}

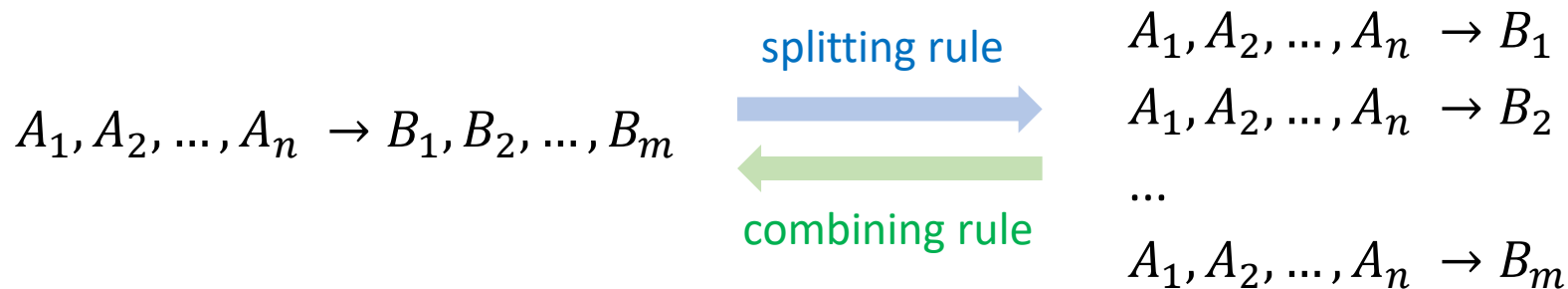
Reasoning about functional dependencies

- We can infer a FD from other given FD's
- e.g., assume $R(A,B,C)$ satisfies the FD's $A \rightarrow B$ and $B \rightarrow C$
 - we can deduce that R also satisfies the FD $A \rightarrow C$
 - assume two tuples (a,b_1,c_1) and (a,b_2,c_2) agree on attribute A
 - since R satisfies $A \rightarrow B$, they must also agree on B
 - i.e., $b_1 = b_2$, and so the tuples are really (a,b,c_1) and (a,b,c_2)
 - similarly, since R satisfies $B \rightarrow C$, they must agree on C
 - i.e., $c_1 = c_2$, and so the tuples are really (a,b,c) and (a,b,c)
 - we have proved that any two tuples of R that agree on A also agree on C
 - i.e., FD $A \rightarrow C$
- The above rule is the **transitive rule**

Equivalence of two sets of FD's

- Two sets of FD's S and T are **equivalent**
 - if the set of relation instances satisfying S is exactly the same as
 - the set of relation instances satisfying T
- A set of FD's S **follows from** a set of FD's T
 - if every relation instance that satisfies all the FD's in T
 - also satisfies all the FD's in S
- Two sets of FD's S and T are equivalent
 - if and only if S follows from T , and T follows from S
- There are **rules** about FD's that can
 - replace a set of FD's by an equivalent set, or
 - add new rules that follow from a set of FD's to the original set
 - e.g., transitive rule

Splitting / combining rule



- There is no splitting rule for left sides
- e.g., *title year* $\rightarrow$ *length* for Movies1
 - if we try to split the left side, then we get two false FD's

title $\rightarrow$ *length*

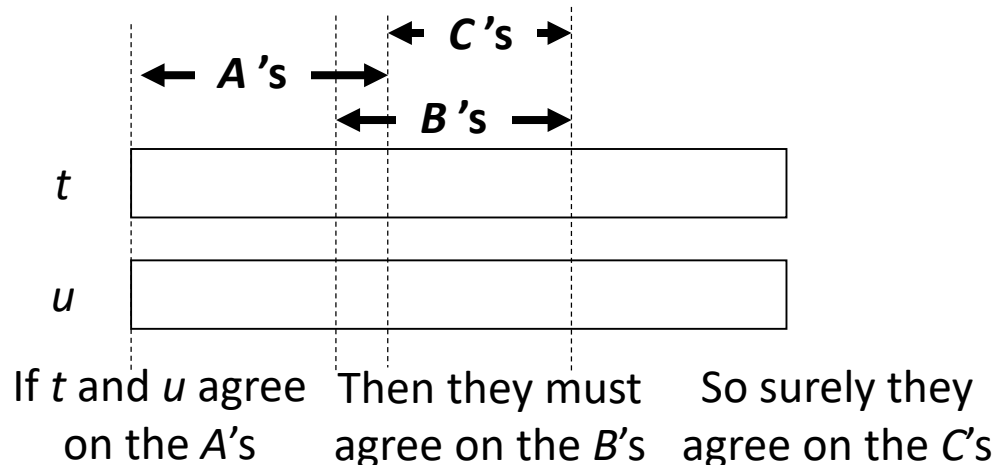
year $\rightarrow$ *length*

- there can be several movies with the same title or year
 - i.e., each of them does not functionally determine length
- c.f., combining rule works for left sides

Trivial functional dependencies

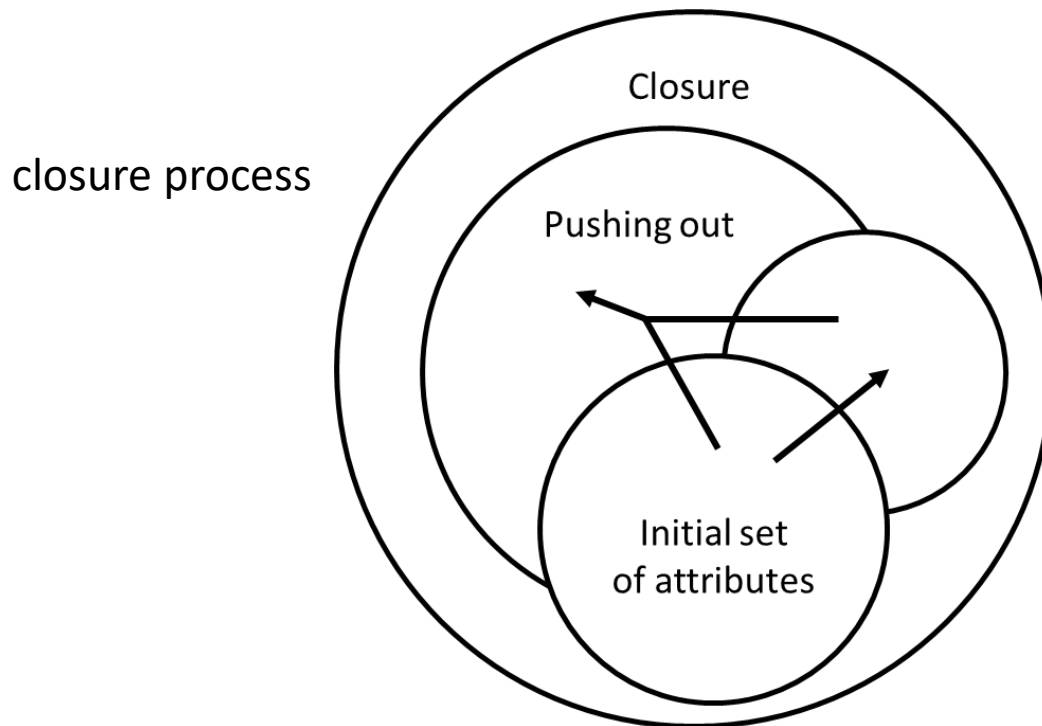
- $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ is trivial
 - if $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots, A_n\}$
- Non-trivial FD
 - some (but **not all**) attributes on right of FD are also on left
 - it can be simplified by removing common attributes from right
 - e.g., $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ is equivalent to

$A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ where the C's are all those B's that are not also A's



Computing the closure of attributes

- Closure of $\{A_1, A_2, \dots, A_n\}$ under a set of FD's S
 - the set of attributes C s.t. every relation that satisfies all the FD's in S also satisfies $A_1, A_2, \dots, A_n \rightarrow C$
 - i.e., the set of attributes C s.t. $A_1, A_2, \dots, A_n \rightarrow C$ follows from the FD's of S
 - we denote the closure as $\{A_1, A_2, \dots, A_n\}^+$



Algorithm for computing the closure

- Input: a set of attributes $\{A_1, A_2, \dots, A_n\}$, a set of FD's S
- Output: the closure $\{A_1, A_2, \dots, A_n\}^+$
- Steps
 - ① if necessary, split the FD's of S , so each FD in S has a single attribute on the right
 - ② let X be a set of attributes that eventually will become the closure; initialize X to be $\{A_1, A_2, \dots, A_n\}$
 - ③ repeatedly search for some FD
$$B_1, B_2, \dots, B_m \rightarrow C$$
such that all of $B_1, B_2, \dots, B_m$ are in the set of attributes X , but C is not; add C to the set X
 - ④ after no more attributes can be added to it, output the set X

Example

- Relation has attributes $\{A, B, C, D, E, F\}$ and following FD's $\{AB \rightarrow C, BC \rightarrow AD, D \rightarrow E, CF \rightarrow B\}$
- What is $\{A, B\}^+$?
 - split $BC \rightarrow AD$ into $BC \rightarrow A$ and $BC \rightarrow D$
 - start with $X = \{A, B\}$
 - X becomes $\{A, B, C\}$ due to the FD $AB \rightarrow C$
 - X becomes $\{A, B, C, D\}$ due to the FD $BC \rightarrow D$
 - X becomes $\{A, B, C, D, E\}$ due to the FD $D \rightarrow E$
 - no more changes to X ; so $\{A, B\}^+ = \{A, B, C, D, E\}$

Test whether a FD follows from a set of FD's

- FD $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ follows from a set of FD's S

if and only if

all of $B_1, B_2, \dots, B_m$ are in $\{A_1, A_2, \dots, A_n\}^+$

– $\{A_1, A_2, \dots, A_n\}^+$ is computed using S

- e.g., $S = \{AB \rightarrow C, BC \rightarrow AD, D \rightarrow E, CF \rightarrow B\}$
 - $AB \rightarrow D$ follows from S , since $\{A, B\}^+ = \{A, B, C, D, E\}$
 - $D \rightarrow A$ does not follow from S , since $\{D\}^+ = \{D, E\}$

- The closure algorithm discovers all true FD's (proof in book)

Transitive rule

- Rule

- if $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ and
- $B_1, B_2, \dots, B_m \rightarrow C_1, C_2, \dots, C_k$ hold in relation R,
- then $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ also holds in R

- Proof

- to test whether $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ holds
- we compute $\{A_1, A_2, \dots, A_n\}^+$ w.r.t. the two given FD's
- $\{A_1, A_2, \dots, A_n\}^+$ becomes $\{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m\}$ and
- again becomes $\{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k\}$
- all the C's are in $\{A_1, A_2, \dots, A_n\}^+$
- thus, $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ holds

Choosing sets of functional dependencies

- We have a choice of which FD's we use to represent the full set of FD's for a relation
 - if we are given a set of FD's S , then any set of FD's equivalent to S becomes a basis for S
 - we consider only bases whose FD's have singleton right sides, to avoid explosion of possible bases
- **Minimal basis**: a basis B that satisfies three conditions
 - ① all the FD's in B have singleton right sides
 - ② if any FD is removed from B , the result is no longer a basis
 - ③ if we remove one or more attributes from the left side of any FD in B , the result is no longer a basis

Example of minimal bases

- $R(A,B,C)$
 - assume that each attribute functionally determines the other two attributes
 - i.e., $S = \{A \rightarrow B, A \rightarrow C, B \rightarrow A, B \rightarrow C, C \rightarrow A, C \rightarrow B\}$
- Many other FD's
 - FD's with two attributes on left: $AB \rightarrow C, AC \rightarrow B, BC \rightarrow A$
 - FD's with more than one attribute on right: $A \rightarrow BC, \dots$
 - trivial FD's: $A \rightarrow A, \dots$
- Minimal bases (multiple minimal bases are possible)
 - $\{A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow B\}$
 - $\{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$

Projecting functional dependencies

- Inputs
 - a relation R ; a set of FD's S that hold in R
 - a second relation R_1 computed by the projection $R_1 = \pi_L(R)$
- Output: the set of FD's that hold in R_1
- Steps
 - ① let T be the eventual output set of FD's (initially empty)
 - ② add all nontrivial FD's $X \rightarrow A$ to T s.t. A is both in X^+ and R_1
 - X is a subset of the attributes of R_1
 - we compute X^+ for each X (w.r.t. the set of FD's S)
 - ③ now, T is a basis for the FD's that hold in R_1 , but may not be minimal; construct a minimal basis
 - an FD F in T follows from the other FD's in T ; remove F from T

Example

- Suppose $R(A, B, C, D)$ has FD's $A \rightarrow B$, $B \rightarrow C$, and $C \rightarrow D$
- Project out the attribute B , leaving a relation $R_1(A, C, D)$
- Find the FD's for R_1
 - $\{A\}^+ = \{A, B, C, D\}$, thus $A \rightarrow C$, $A \rightarrow D$ hold in R_1
 - $\{C\}^+ = \{C, D\}$, thus $C \rightarrow D$ holds in R_1
 - $\{D\}^+ = \{D\}$, thus no nontrivial FD holds in R_1
 - $\{A, C\}^+ = \{A, B, C, D\}$, thus $AC \rightarrow D$ hold in R_1 (do not consider)
 - $\{C, D\}^+ = \{C, D\}$, thus no nontrivial FD holds in R_1
- Minimal basis for the FD's of R_1
 - FD's discovered: $A \rightarrow C$, $A \rightarrow D$ and $C \rightarrow D$
 - remove $A \rightarrow D$ due to its transitivity
 - the set of FD's that hold in R_1 becomes $\{A \rightarrow C, C \rightarrow D\}$

Designing relational database schemas

Design of relational database schemas

- Motivating example

- many attribute values are repeated, for each star of the movie
- repetition of the information is **redundant**
- redundancy may introduce **several kinds of anomalies**

title	year	length	genre	studioName	starName
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With the Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

Movies1

Anomalies

- Redundancy
 - information may be repeated unnecessarily in multiple tuples
- Update anomalies
 - we may change information in one tuple but leave the same information unchanged in another tuple
 - e.g., carelessly change the length in the 1st tuple, but not 2nd and 3rd tuples
- Deletion anomalies
 - if a set of values becomes empty, we may lose other information as a side effect
 - e.g., we should delete “Vivien Leigh”, but the last tuple for “Gone with the wind” would also disappear

Decomposing relations

- Decomposing relations can eliminate the anomalies
- Given a relation $R(A_1, A_2, \dots, A_n)$
 - we decompose into $S(B_1, B_2, \dots, B_m)$ and $T(C_1, C_2, \dots, C_k)$ s.t.

$$1. \{A_1, A_2, \dots, A_n\} = \{B_1, B_2, \dots, B_m\} \cup \{C_1, C_2, \dots, C_k\}$$

$$2. S = \pi_{B_1 B_2, \dots, B_m}(R)$$

$$3. T = \pi_{C_1 C_2, \dots, C_k}(R)$$

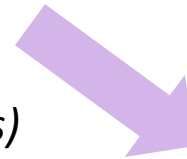
Example

Movies1

title	year	length	genre	studioName	starName
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With the Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers



*decomposition
(eliminates the anomalies)*



Movies2

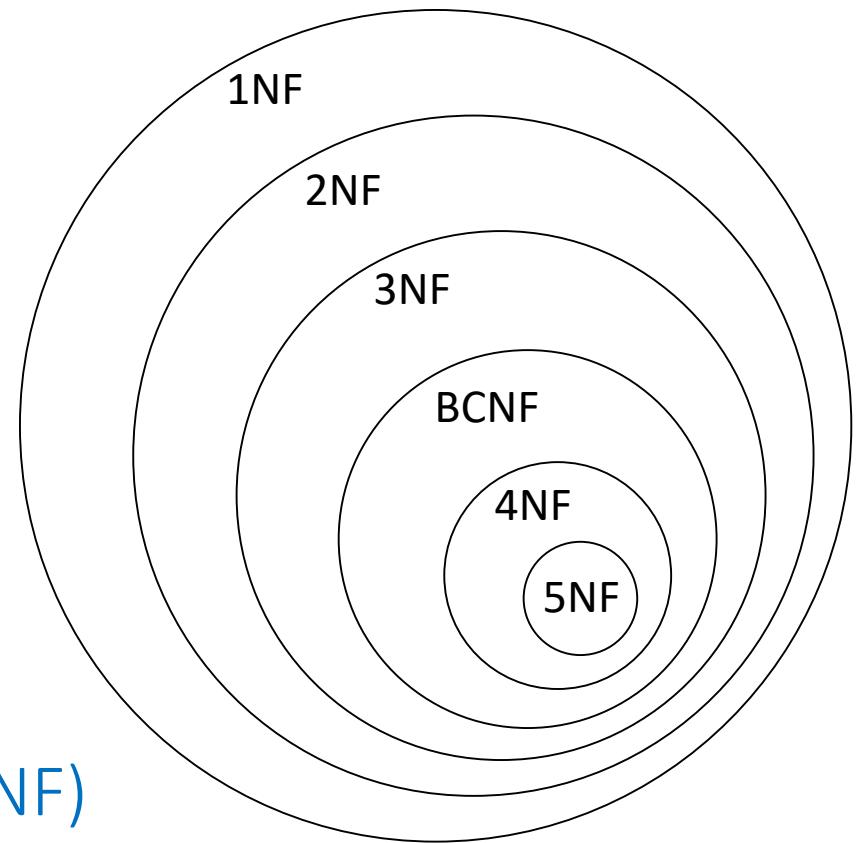
<u>title</u>	<u>year</u>	length	genre	studioName
Star Wars	1977	124	SciFi	Fox
Gone With the Wind	1939	231	drama	MGM
Wayne's World	1992	95	comedy	Paramount

Movies3

<u>title</u>	<u>year</u>	<u>starName</u>
Star Wars	1977	Carrie Fisher
Star Wars	1977	Mark Hamill
Star Wars	1977	Harrison Ford
Gone With the Wind	1939	Vivien Leigh
Wayne's World	1992	Dana Carvey
Wayne's World	1992	Mike Meyers

Normal forms

- First normal form (1NF)
- Second normal form (2NF)
- Third normal form (3NF)
- Boyce-Codd normal form (BCNF)
- Fourth normal form (4NF)
 - remove multivalued dependency (MVD)
- Fifth normal form (5NF)



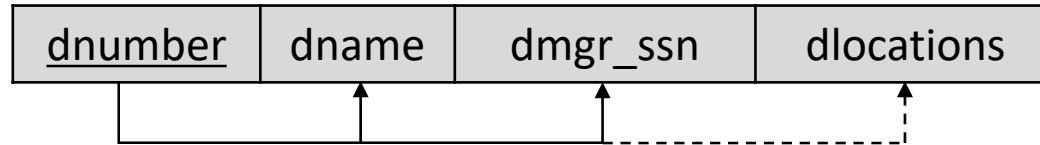
First normal form (1NF)

- Unnest into a set of flat relations
- Does **not allow nested relations**
 - only attribute values permitted are **single atomic values**
 - **non-atomic value (e.g., set, list, ...) are not allowed**
- To change a relation R to 1NF
 - remove nested relation attributes into a new relation
 - propagate the primary key of R into it
 - use several atomic attributes

Example

EXTRA

DEPARTMENT



DEPARTMENT

<u>dnumber</u>	dname	dmgr_ssn	dlocations
5	Research	333445555	{Bellaire, Sugarland, Houston}
4	Administration	987654321	{Stafford}
1	Headquarters	888665555	{Houston}

NOT 1NF

DEPARTMENT

<u>dnumber</u>	dname	dmgr_ssn
5	Research	333445555
5	Research	333445555
5	Research	333445555
4	Administration	987654321
1	Headquarters	888665555

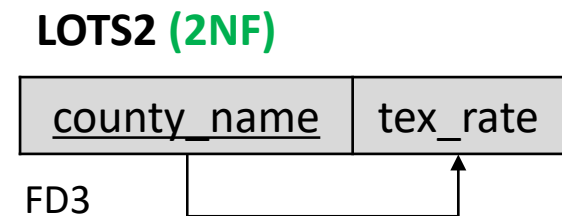
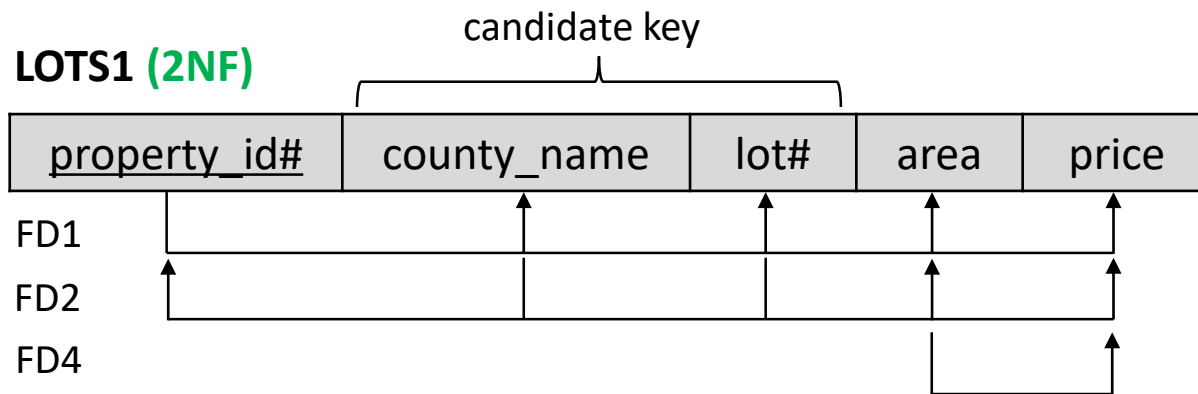
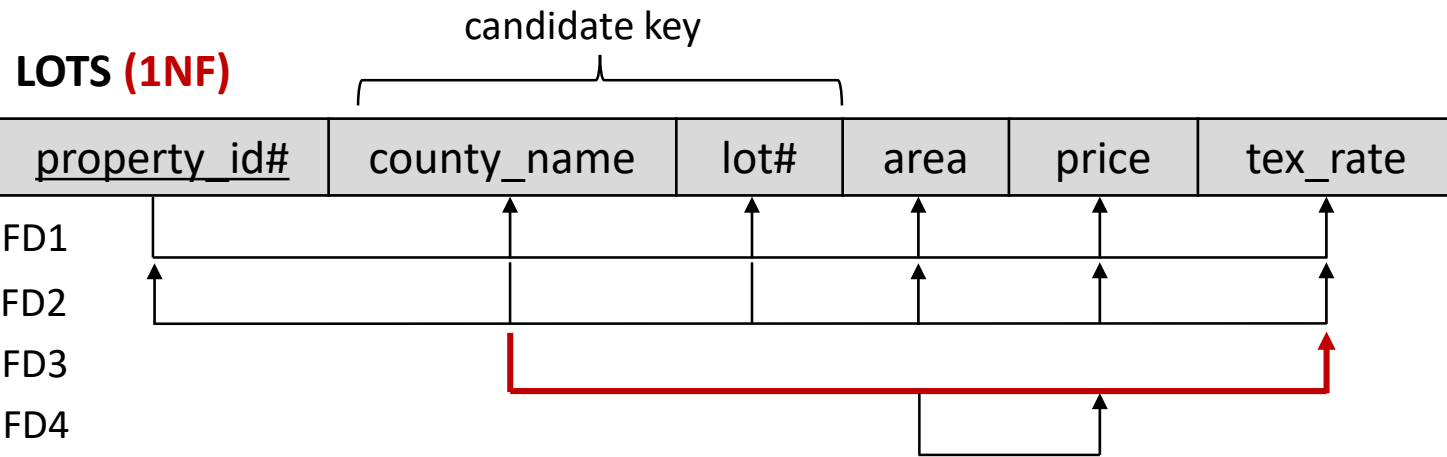
DLOCATION

<u>dnumber</u>	dlocation
5	Bellaire
5	Sugarland
5	Houston
4	Stafford
1	Houston

1NF

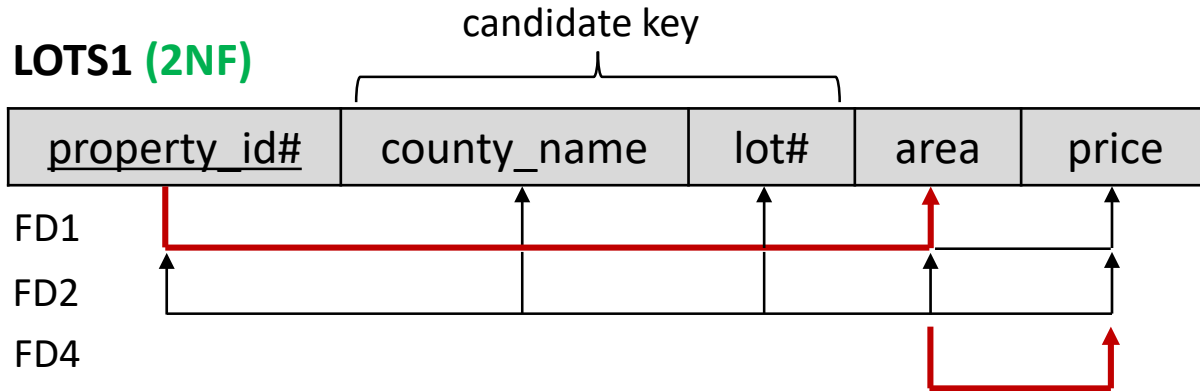
Second normal form (2NF)

- R is in 2NF if **every nonprime attribute** of R is **not partially dependent** on **any key** of R
 - prime attribute: a member of some candidate key of R

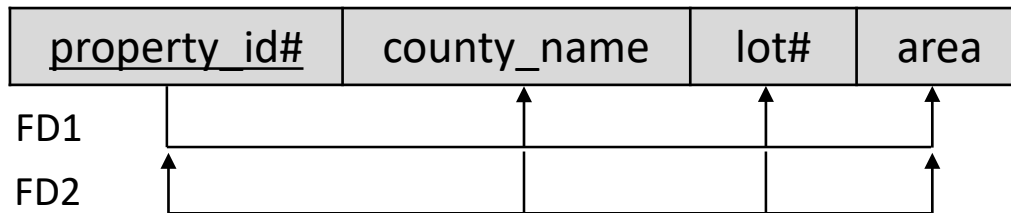


Third normal form (3NF)

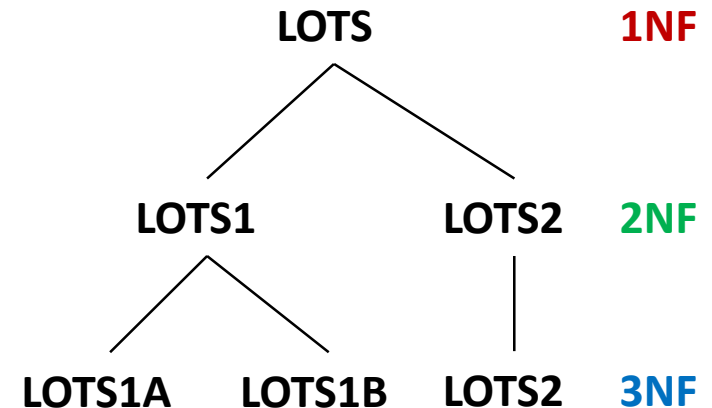
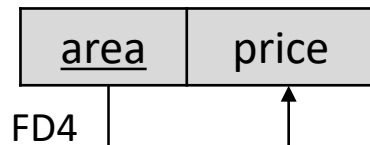
- R is in 3NF if it satisfies 2NF, and **no nonprime attribute** of R is **transitively dependent** on the primary key of R



LOTS1A (3NF)

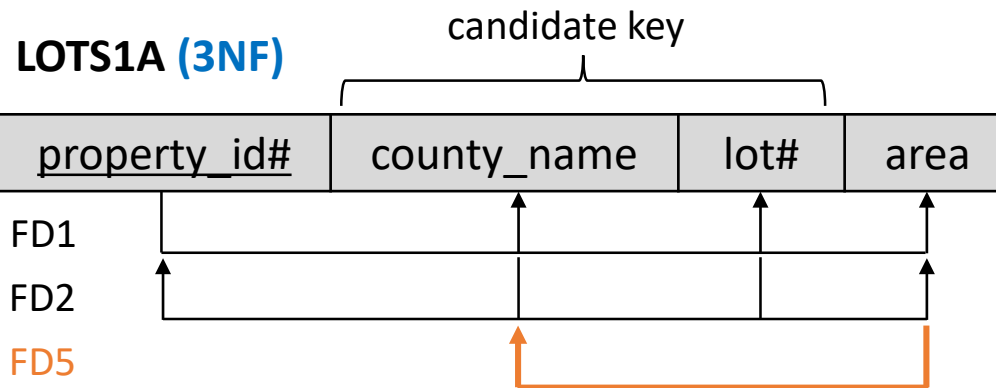


LOTS1B (3NF)

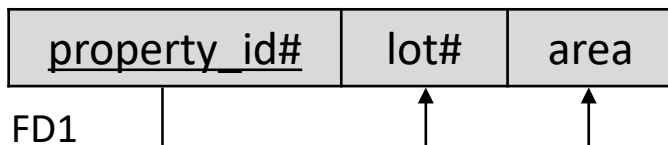


Boyce-Codd normal form (BCNF)

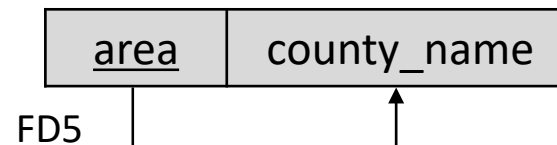
- R is in BCNF if
 - whenever a nontrivial FD $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ holds in R
 - then, $\{A_1, A_2, \dots, A_n\}$ is a superkey for R(i.e., left side of every nontrivial FD must be a superkey)



LOTS1AX (BCNF)



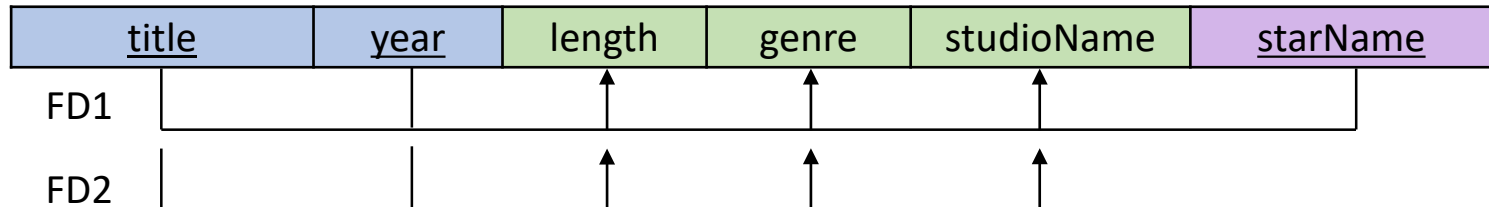
LOTS1AY (BCNF)



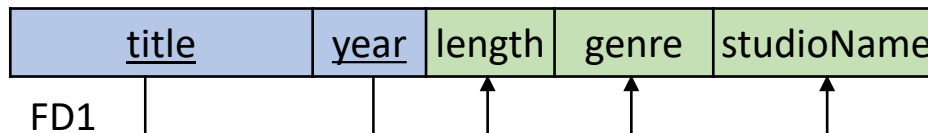
Example

- Movies1 is not in BCNF
 - {title, year, starName} is the only key for Movies1
 - the FD *title year → length genre studioName* holds
 - left side of the FD is not a superkey
 - so, the existence of the FD violates the BCNF condition

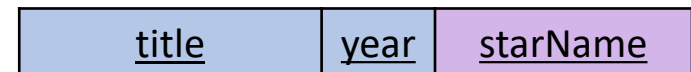
Movies1



Movies2



Movies3



no nontrivial FD's

- Movies2 is in BCNF
 - the FD *title year* → *length genre studioName* holds
 - the only key for Movies2 is {title, year}
 - the only nontrivial FD's must have at least {title, year} on left
 - therefore, their left sides must be superkeys

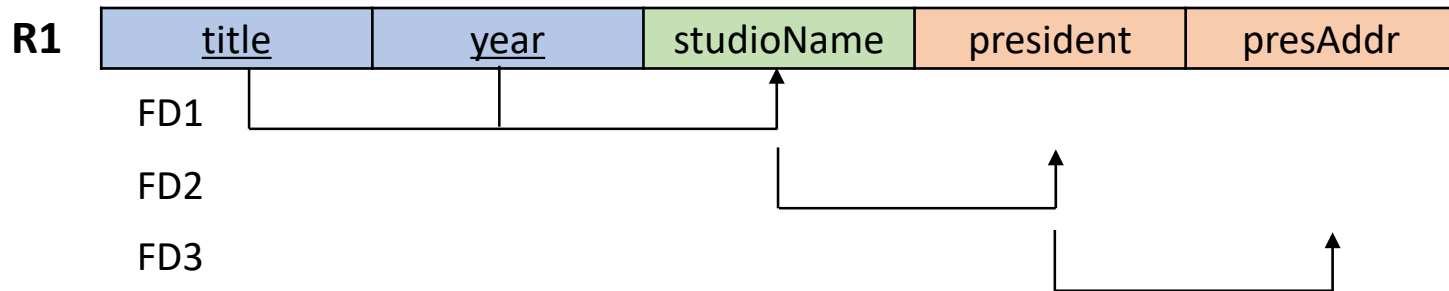
Movies2

title	year	length	genre	studioName
Star Wars	1977	124	SciFi	Fox
Gone With the Wind	1939	231	drama	MGM
Wayne's World	1992	95	comedy	Paramount

- Movies3 is also in BCNF
 - because there is no nontrivial FD's
- Decomposed Movies1 into Movies2 and Movies3 in BCNF
 - we can reconstruct the original relation instance exactly from the decomposed relation instances (by using join)

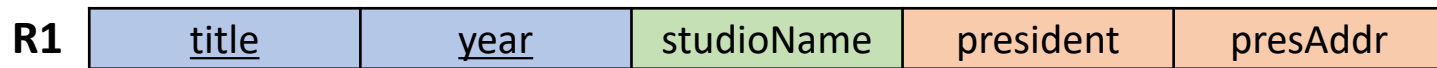
Example

- president: the president of a studio
- presAddr: the address of the president of the studio

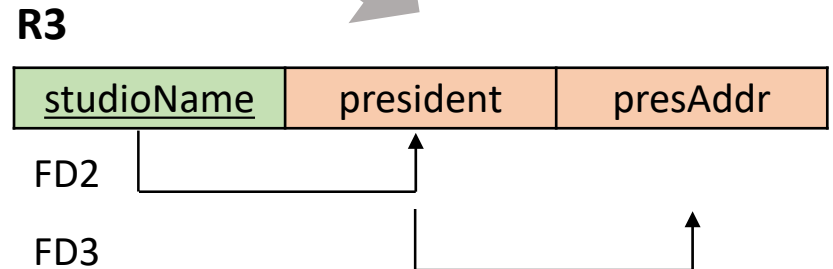
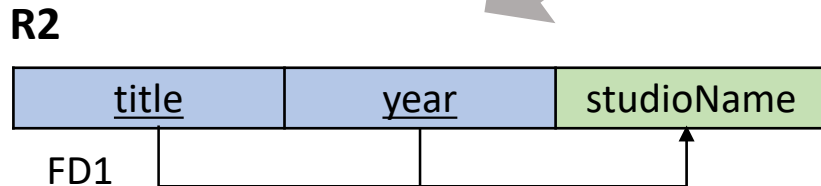


- $\{\text{title}, \text{year}\}$ is the only key for the relation
 - $\{\text{title}, \text{year}\}^+ = \{\text{title}, \text{year}, \text{studioName}, \text{president}, \text{presAddr}\}$
- Thus, FD2 and FD3 violate BCNF

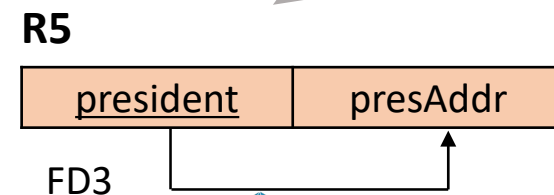
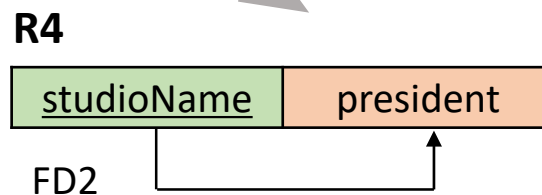
- We decompose the relation based on FD2
 - the sole key of R2 is {title, year}
 - the sole key of R3 is {studioName}, but also has FD3
 - FD3 violates BCNF
 - thus, we must decompose R3 again into R4 and R5 in BCNF



decomposition



decomposition



BCNF decomposition algorithm

- Input: a relation R_0 with a set of FD's S_0
- Output: a collection of decomposed relations in BCNF
- Steps: (initially $R=R_0$ and $S=S_0$)
 - ① check whether R is in BCNF; if so, return $\{R\}$ as the answer
 - ② if there are BCNF violation, let one be $X \rightarrow Y$;
compute X^+ (using Algorithm in p.18);
choose $R_1 = X^+$ as one relation schema;
let R_2 have attributes X and the attributes of R that are not in X^+
 - ③ compute the sets of FD's for R_1 and R_2 (using Algorithm in p.24);
let these be S_1 and S_2 , respectively
 - ④ recursively decompose R_1 and R_2 ; return the union of the results

Three distinct properties of decomposition

① Elimination of anomalies (by decomposition)

- redundancy, update anomalies, deletion anomalies

② Recoverability of information

- can we recover the original relation from the tuples in its decomposition?

③ Preservation of dependencies

- when we reconstruct the original relation from the decomposition by joining, the result will satisfy the original FD's?

- BCNF gives us (1) and (2) (e.g., p.30, p.38)

- but does not necessarily give us all three

Recovering information from a decomposition

- Lossless join

- getting all the original tuples of a relation R back by (natural) joining R's decomposed relations

- e.g., $R(A,B,C)$, an FD $B \rightarrow C$ that is a BCNF violation

- we decompose R into relations $R_1(A,B)$ and $R_2(B,C)$

- assumption: there are only two tuples of R, $t = (a,b,c)$, $v = (d,b,e)$

R_1	A	B	$\bowtie$	B	C	R_2
	a	b		b	c	
	d	b		b	e	

- natural join $R_1 \bowtie R_2$ can generate a *bogus* tuple $x = (a,b,e)$? no

- FD means any two tuples that agree in B must also agree in C

- since t and v agree in B, they also agree on C (i.e., $c = e$)

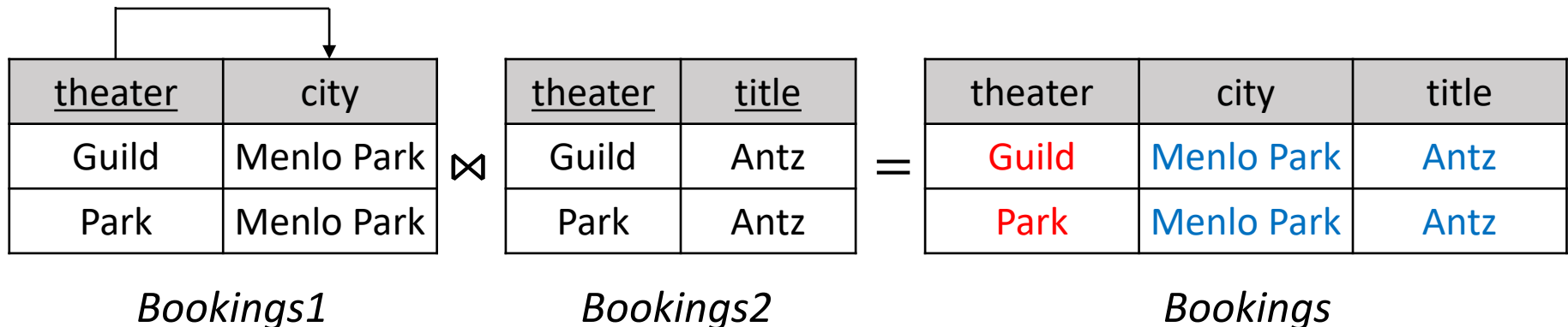
- tuple $x = (a,b,e)$ is really (a,b,c) , i.e., $x = t$

- The argument in the previous slide works in general
 - assumption: X , Y , and Z are any sets of attributes, $X \cup Y \cup Z = R$
 - if $Y \rightarrow Z$ holds in R ,
 - then, $\pi_{X \cup Y}(R) \bowtie \pi_{Y \cup Z}(R) = \pi_{X \cup Y \cup Z}(R) = R$
- We may conclude
 - if we decompose a relation according to Algorithm in p.37,
 - then the original relation can be recovered exactly by the natural join

Dependency preservation

- e.g., **Bookings(title, theater, city)**
 - title: the name of a movie
 - theater: the name of a theater where the movie is being shown
 - city: the city where the theater is located
 - a tuple (m,t,c)
 - the movie with title m is currently being shown at theater t in city c
 - we assert the FD's: *theater* $\rightarrow$ *city*, *title city* $\rightarrow$ *theater*
 - no single attribute is a key
 - candidate keys: {title, city}, {theater, title}
 - FD *theater* $\rightarrow$ *city* violates BCNF

- Bookings(title, theater, city)
 - FD's: *theater* → *city*, *title city* → *theater*
 - candidate keys: {*title, city*}, {*theater, title*}
- Decomposition 1: {theater, city}, {theater, title}
 - *theater* → *city* is satisfied (decomposition based on that FD)
 - *Bookings1* and *Bookings2* are permissible according to the FD
 - but, when we join both relations,
 - the resulting relation violates the FD *title city* → *theater*



Three distinct properties of decomposition (revisited)

① Elimination of anomalies (by decomposition)

- redundancy, update anomalies, deletion anomalies

② Recoverability of information

- can we recover the original relation from the tuples in its decomposition?

③ Preservation of dependencies

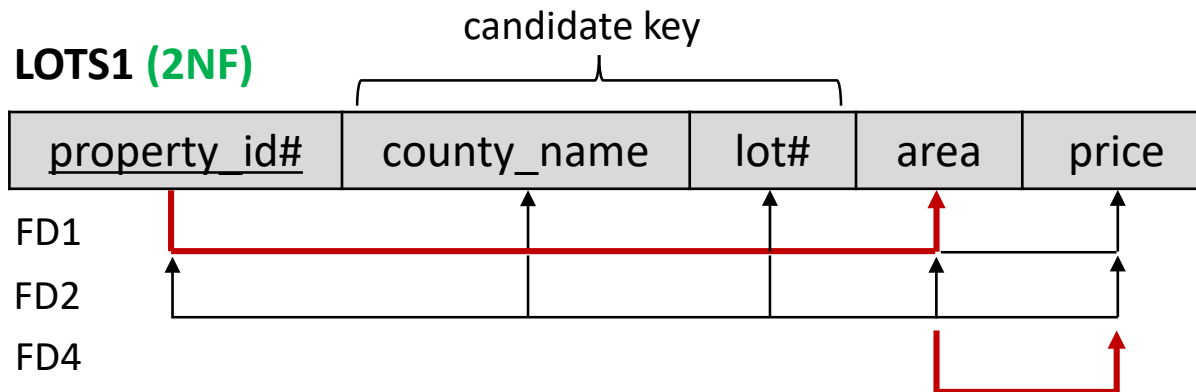
- when we reconstruct the original relation from the decomposition by joining, the result will satisfy the original FD's?

- 3NF gives us (2) and (3)

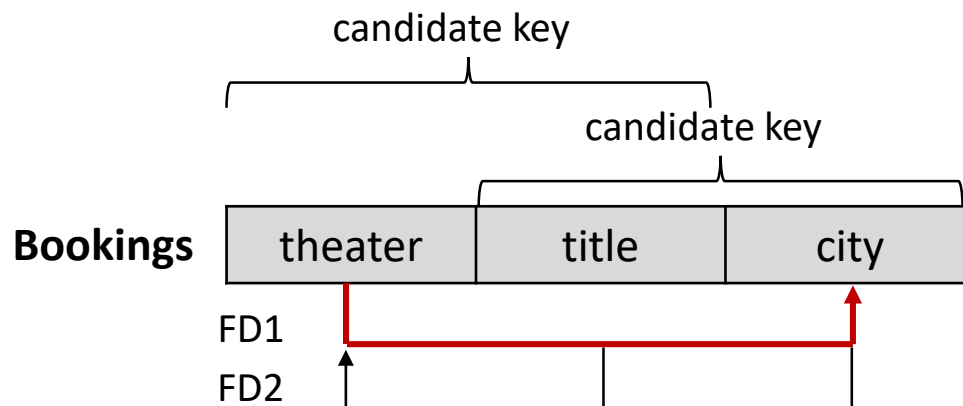
- but does not necessarily give us all three

Third normal form (3NF)

- Problem about dependency preservation of BCNF
 - can be solved by relaxing BCNF requirement slightly
 - i.e., by using 3NF instead of BCNF
- R is in 3NF if it satisfies 2NF and **no nonprime attribute** of R is **transitively dependent** on the primary key of R
 - i.e., for each nontrivial FD,
 - either the left side is a superkey,
 - or the right side consists of prime attributes only



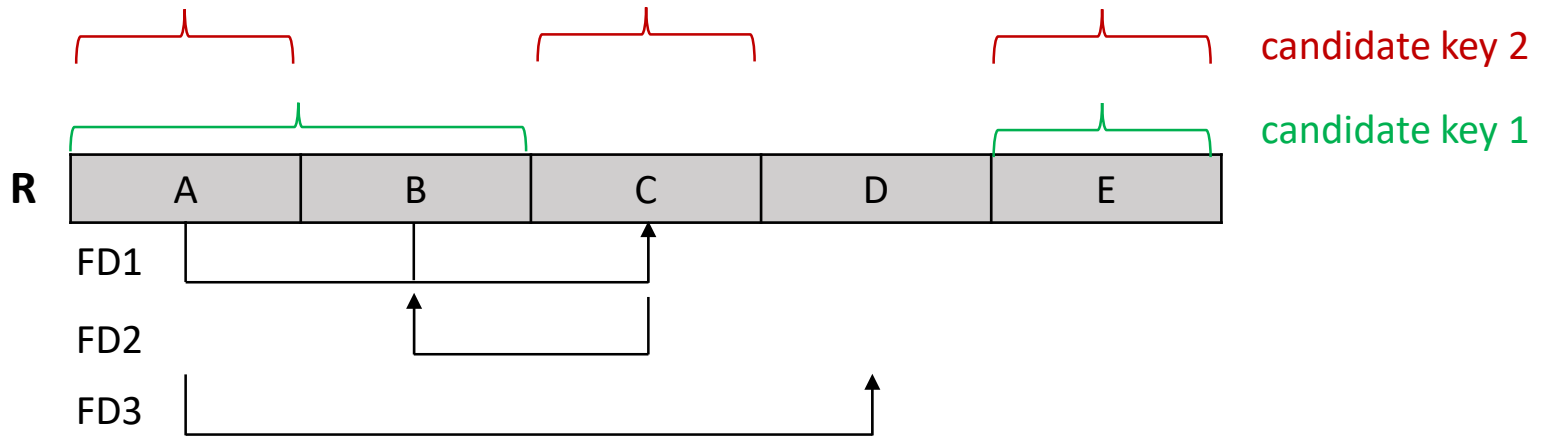
- Bookings(title, theater, city)
 - FD's: *theater* → *city*, *title city* → *theater*
 - candidate keys: {*title, city*}, {*theater, title*}
 - FD *theater* → *city* violates BCNF
- But, FD *theater* → *city* is allowed in 3NF
 - because the right side, city, is a prime attribute
- Thus, Bookings satisfies 3NF without being decomposed
 - FD's {*theater* → *city*, *title city* → *theater*} are preserved
 - instead, some anomalies may exist



3NF decomposition algorithm

- Input: a relation R with a set of FD's F
- Output: a collection of decomposed relations in 3NF
- Steps:
 - ① find a minimal basis for F , say G
 - ② for each FD $X \rightarrow A$ in G , use XA as the schema of one of the relations in the decomposition
 - ③ if none of the relation schemas from Step (2) is a superkey for R , add another relation whose schema is a key for R

Example



- Step 1: the set of FD's is already minimal basis for **F**
- Step 2: decomposed into $S_1(A, B, C)$, $S_2(B, C)$, $S_3(A, D)$
- Step 3: **R** has two keys $\{A, B, E\}$, $\{A, C, E\}$
 - add one of them, say $S_4(A, B, E)$

Multivalued dependencies (MVD)

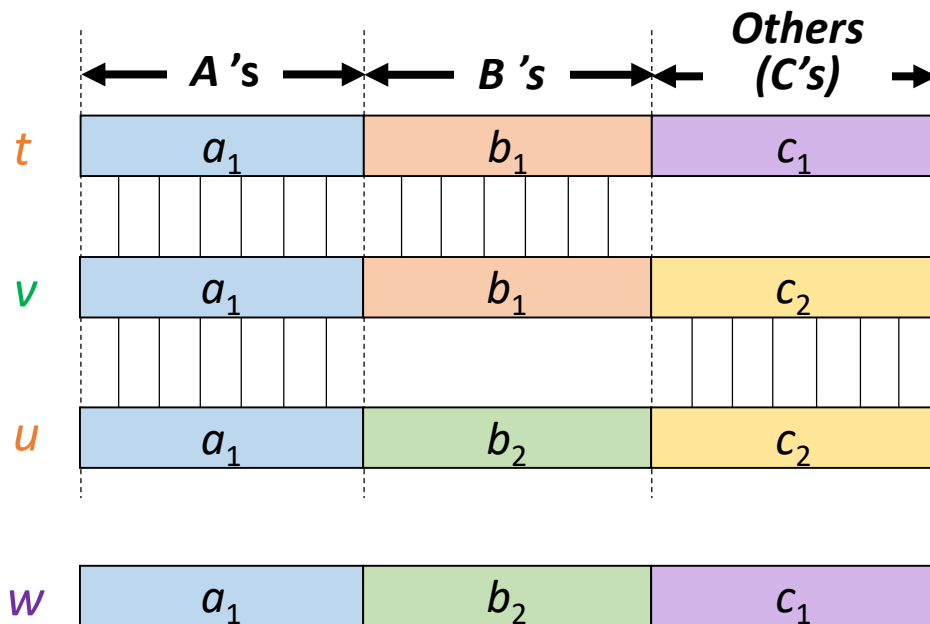
- Relations in BCNF may still have a kind of redundancy
- e.g.,
 - star may have several addresses
 - star usually has a set of titles and years of movies in which the star appeared

name	address		movie	
	street	city	title	year
C. Fisher	123 Maple St.	Hollywood	Star Wars	1977
C. Fisher	5 Locust Ln.	Malibu	Star Wars	1977
C. Fisher	123 Maple St.	Hollywood	Empire Strikes Back	1980
C. Fisher	5 Locust Ln.	Malibu	Empire Strikes Back	1980
C. Fisher	123 Maple St.	Hollywood	Return of the Jedi	1983
C. Fisher	5 Locust Ln.	Malibu	Return of the Jedi	1983

- There is obvious redundancy
 - address and movie facts are repeated in all combinations
- Yet there is no BCNF violation
 - no nontrivial FD's at all
 - e.g., city is not functionally determined by the other four attributes
 - you can check that none of the five attributes is functionally determined by the other four attributes
 - all five attributes form the only key

Definition of MVD

- $A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$ holds for a relation R if
 - for each pair of tuples t and u of relation R that agree on all the A's, we can find in R some tuple v that agrees:
 - ① with both t and u on the A's,
 - ② with t on the B's, and
 - ③ with u on all attributes of R that are not in the A's or B's



Example

- MVD 1: *name* → *street city*
- MVD 2: *name* → *title year*

<i>A's</i>		<i>B's (or C's)</i>		<i>C's (or B's)</i>	
name	street	city	title	year	
C. Fisher	123 Maple St.	Hollywood	Star Wars	1977	
C. Fisher	5 Locust Ln.	Malibu	Star Wars	1977	
C. Fisher	123 Maple St.	Hollywood	Empire Strikes Back	1980	
C. Fisher	5 Locust Ln.	Malibu	Empire Strikes Back	1980	
C. Fisher	123 Maple St.	Hollywood	Return of the Jedi	1983	
C. Fisher	5 Locust Ln.	Malibu	Return of the Jedi	1983	

Fourth normal form (4NF)

- For eliminating the redundancy caused by MVD's
- The condition is **essentially the BCNF condition**, but applied to **MVD's instead of FD's**
- R is in 4NF if
 - whenever a nontrivial **MVD $A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$** holds in R
 - then, $\{A_1, A_2, \dots, A_n\}$ is a superkey for R
- e.g., relation in the previous slide violates 4NF condition
 - **name is not a superkey in $\text{name} \twoheadrightarrow \text{street city}$**
- **4NF is truly a generalization of BCNF**
 - every BCNF violation is also 4NF violation
 - every relation in 4NF is in BCNF

4NF decomposition algorithm

- Input: a relation R_0 with a set of FD's and MVD's S_0
- Output: a collection of decomposed relations in 4NF
- Steps: (initially $R=R_0$ and $S=S_0$)
 - ① check whether R is in 4NF; if so, return $\{R\}$ as the answer
 - ② if there are 4NF violation, say $A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$, break the schema into two schemas
 - R_1 , whose schema is the A 's and B 's
 - R_2 , whose schema is the A 's and C 's
 - ③ find FD's and MVD's that hold in R_1 and R_2
 - ④ recursively decompose R_1 and R_2

name	street	city	title	year
C. Fisher	123 Maple St.	Hollywood	Star Wars	1977
C. Fisher	5 Locust Ln.	Malibu	Star Wars	1977
C. Fisher	123 Maple St.	Hollywood	Empire Strikes Back	1980
C. Fisher	5 Locust Ln.	Malibu	Empire Strikes Back	1980
C. Fisher	123 Maple St.	Hollywood	Return of the Jedi	1983
C. Fisher	5 Locust Ln.	Malibu	Return of the Jedi	1983



name	street	city
C. Fisher	123 Maple St.	Hollywood
C. Fisher	5 Locust Ln.	Malibu

R1

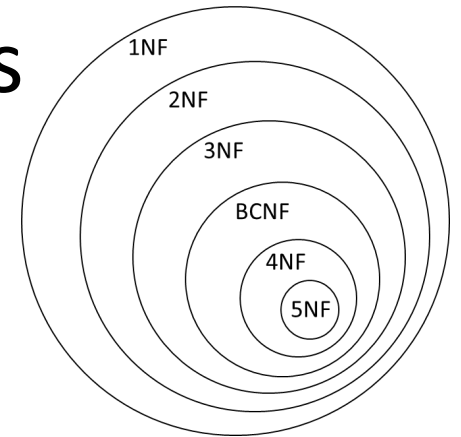
name	title	year
C. Fisher	Star Wars	1977
C. Fisher	Empire Strikes Back	1980
C. Fisher	Return of the Jedi	1983

R2



name	street	city	title	year
C. Fisher	123 Maple St.	Hollywood	Star Wars	1977
C. Fisher	5 Locust Ln.	Malibu	Star Wars	1977
C. Fisher	123 Maple St.	Hollywood	Empire Strikes Back	1980
C. Fisher	5 Locust Ln.	Malibu	Empire Strikes Back	1980
C. Fisher	123 Maple St.	Hollywood	Return of the Jedi	1983
C. Fisher	5 Locust Ln.	Malibu	Return of the Jedi	1983

Relationships among normal forms



Property	3NF	BCNF	4NF
Eliminate redundancy due to FD's	No	Yes	Yes
Eliminate redundancy due to MVD's	No	No	Yes
Preserves FD's	Yes	No	No
Preserves MVD's	No	No	No

Thank you

Q & A